We need to do better if we would like to keep our tree balanced and its performance acceptable: and in section 10.6 we will see a possible solution, Ss$^+$-trees.

10.4 Similarity Search

Before discussing how to improve the balancing of Ss-trees, we can finish our discussion on its methods; so far, we have seen how to construct such a tree, but what can we use it for? Not surprisingly, nearest neighbor search is one of the main applications of this data structure, you probably had guessed it. Range search, like for k-d trees, is another important application for Ss-trees; both NN and range searches fall into the category of similarity search, queries on large multi-dimensional spaces where our only criterion is the similarity between a pair of objects.

As we have discussed for k-d trees in chapter 9, Ss-trees can also (and more easily) be extended to support approximated similarity search: if you remember, at the end of chapter 9, in section 9.4, we have mentioned that approximate queries are a possible solution to k-d trees performance issues; we'll talk more diffusely about these methods in section 10.4.2.

10.4.1 Nearest Neighbor Search

The nearest-neighbor search algorithm is similar to what we have seen for k-d trees; obviously the tree structure is different, but the main change in the algorithm's logic is the formula we need to use to check if a branch intersects the NN query region (the sphere centered at the query point and